

RED TEAM LEARNING PATH

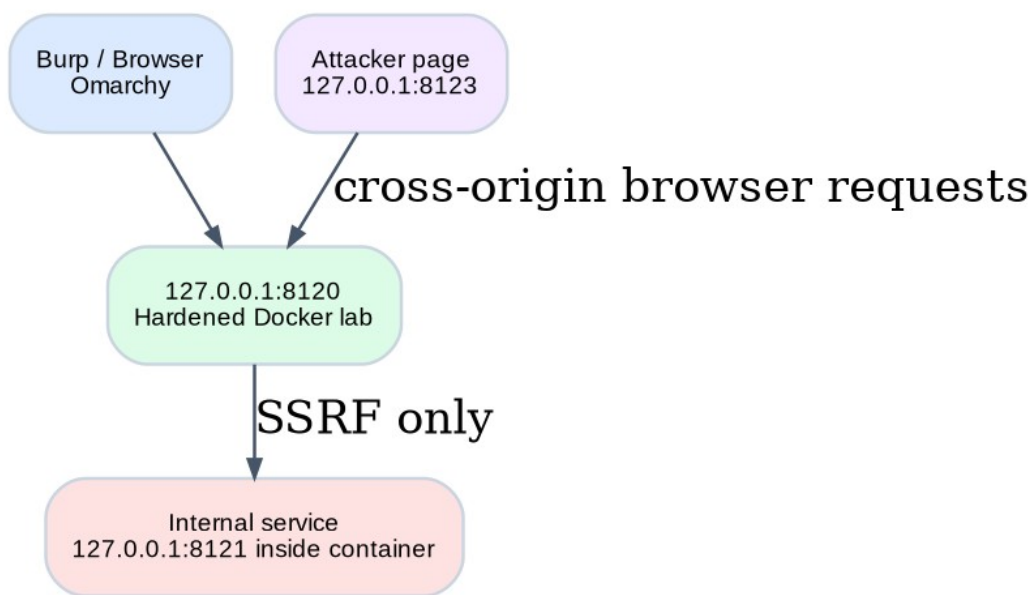
MODULE 12

Web Vulnerabilities II

SSRF - XXE - SSTI - Deserialization - CORS - CSRF - JWT - OAuth - GraphQL - NoSQL - Host Header - Request Smuggling - Cache Poisoning - Race Conditions - Prototype Pollution - Business Logic

AUTHORIZED LABS ONLY

Semua langkah teknis pada modul ini ditujukan untuk local Docker lab Module 12 atau deliberately vulnerable PortSwigger Web Security Academy labs. Jangan menguji website, API, proxy, identity provider, atau infrastructure pihak lain tanpa authorization tertulis.



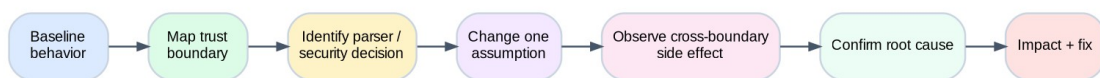
Peta local training lab Module 12

Prerequisites: Module 00-11 - Burp Suite - HTTP/Web fundamentals - Docker

1. Tujuan Pembelajaran

Module 12 membahas vulnerability yang sering muncul ketika beberapa komponen membuat asumsi berbeda tentang request, identity, origin, parser, cache, concurrency, atau internal trust. Target akhirnya bukan hafal exploit chain, tetapi bisa memetakan trust boundary dan membuktikan asumsi keamanan yang rusak.

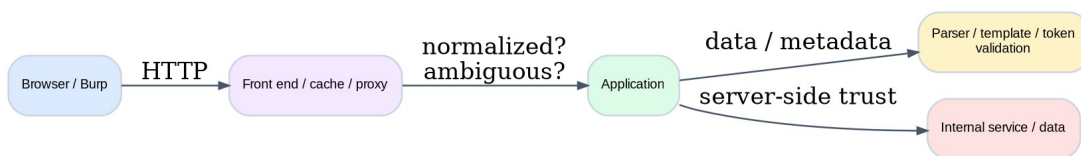
- [] Membedakan browser-origin controls, server-side authorization, dan token validation.
- [] Mengenali SSRF, XXE, dan SSTI sebagai server-side trust/interpreter failures.
- [] Menjelaskan risiko insecure deserialization tanpa membangun gadget berbahaya di luar training lab.
- [] Membedakan CORS dari CSRF dan memahami same-origin vs same-site.
- [] Menguji JWT validation flaw pada local lab dan menjelaskan kenapa claims tidak boleh dipercaya tanpa signature verification.
- [] Memahami OAuth/OIDC, GraphQL, dan NoSQL attack surface melalui guided Academy labs.
- [] Menganalisis Host header, cache key, request parsing, dan parser disagreement.
- [] Menguji race condition dan business-logic flaw secara lokal.
- [] Menulis evidence yang menghubungkan baseline, modified request, side effect, impact, dan remediation.



Advanced web testing workflow: fokus pada trust boundary dan interpretation gap

2. Model Mental: Trust Boundary & Interpretation Gap

Pada module sebelumnya, banyak bug bisa dipahami sebagai input mencapai sink secara tidak aman. Di level advanced, masalah sering terjadi karena dua komponen memandang data yang sama secara berbeda - browser vs server, proxy vs backend, cache vs application, parser vs serializer, atau token issuer vs consumer.



Satu request bisa melewati beberapa komponen dengan asumsi berbeda

Kelas	Boundary yang rusak	Pertanyaan tester
SSRF / XXE	External input -> server-side network/parser	Bisakah user memaksa server mengakses resource yang user sendiri tidak dapat akses?
CORS / CSRF	Other origin -> victim browser -> app	Apakah browser boleh membaca response atau mengirim state-changing request dengan credential?
JWT / OAuth	Token/redirect metadata -> identity decision	Apakah signature, issuer, audience, redirect, state, dan binding diverifikasi?
Request smuggling	Front-end parser -> back-end parser	Apakah kedua komponen sepakat di mana satu request berakhir?
Cache poisoning	Application input -> cache key	Apakah response dipengaruhi input yang tidak menjadi bagian cache key?
Race / logic	Concurrent/atypical state -> business rule	Apakah rule tetap benar saat request paralel atau urutan tidak normal?

Pola pikir utama

Cari asumsi, bukan payload. Tanyakan: komponen mana yang mempercayai apa, siapa yang mengontrol nilai itu, dan apakah komponen berikutnya menafsirkan nilai dengan cara yang sama?

3. Setup Local Advanced Web Lab

Build dan start lab

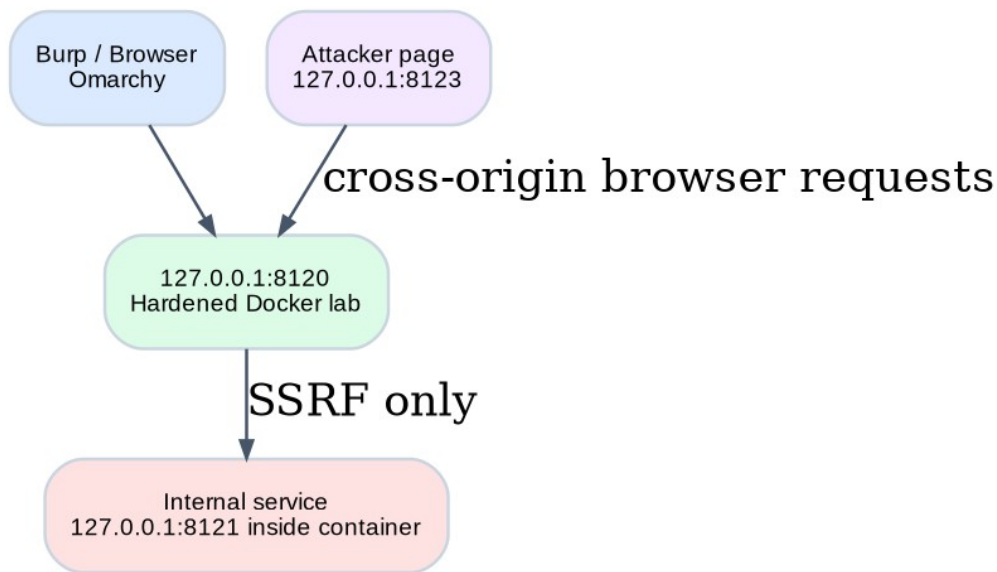
```
cd modules/12-web-vulnerabilities-ii/labs

./workspace-init.sh
./advanced-web-lab.sh build
./advanced-web-lab.sh create
./advanced-web-lab.sh status
```

Aplikasi dipublish hanya ke 127.0.0.1:8120. Internal service untuk SSRF bind di 127.0.0.1:8121 di dalam container dan tidak dipublish ke host. Container dijalankan non-root, cap-drop ALL, no-new-privileges, read-only filesystem, dan tmpfs /tmp.

Safety boundary

Local lab tetap intentionally vulnerable. Jangan mengubah launcher menjadi 0.0.0.0 atau melakukan host volume mount. Cleanup setelah latihan.



Alur local lab: browser/Burp, vulnerable app, internal-only service, dan separate origin

4. SSRF - Server-Side Request Forgery

SSRF terjadi ketika aplikasi membuat request server-side berdasarkan input user tanpa kontrol destination yang memadai. Dampaknya berasal dari perbedaan network reachability: aplikasi sering dapat melihat localhost, metadata service, internal API, atau management plane yang tidak terlihat dari browser user.



SSRF dan XXE dapat menjembatani input eksternal ke resource internal

Hands-on 4.1 - Local SSRF boundary

1. Buka baseline root lab di Burp browser.
2. Kirim request SSRF ke loopback internal service yang sengaja tidak dipublish.
3. Bandingkan dengan request langsung dari host ke port internal; port 8121 tidak dipublish.
4. Catat bahwa server melakukan request atas nama user dan berhasil membaca internal-only response.

Local proof

```
GET /ssrf?url=http://127.0.0.1:8121/internal/admin HTTP/1.1
Host: 127.0.0.1:8120
```

Training guardrail

Endpoint lab hanya mengizinkan hostname localhost / 127.0.0.1. Tujuannya memahami trust boundary, bukan membuat general-purpose SSRF proxy.

Remediation

Gunakan destination allow-list yang sempit, canonical URL parsing, deny private/link-local/loopback destination jika tidak diperlukan, disable redirect surprises, dan tempatkan outbound network controls di level infrastructure.

5. XXE - XML External Entity Injection

XXE muncul ketika XML parser mengizinkan external entity dan aplikasi memproses XML yang dikontrol user. Parser dapat membaca file lokal atau melakukan server-side network access. Banyak library modern mematikan fitur berbahaya secara default, tetapi legacy/configuration mistakes masih penting dipahami.

Hands-on 5.1 - Harmless local file read inside container

XXE proof di disposable container

```
POST /xml HTTP/1.1
Host: 127.0.0.1:8120
Content-Type: application/xml

<?xml version="1.0"?>
<!DOCTYPE demo [<!ENTITY xxe SYSTEM "file:///etc/hostname">]>
<demo>&xxe;</demo>
```

Target proof hanya /etc/hostname di container. Jangan menggunakan secret file atau path host. Observe bahwa value response berasal dari entity expansion, bukan value literal yang dikirim client.

Remediation

Gunakan parser configuration yang menonaktifkan DTD/external entity resolution; prefer simpler data formats bila XML tidak diperlukan; patch library dan validasi parser mode secara eksplisit.

6. SSTI - Server-Side Template Injection

SSTI terjadi saat input user diperlakukan sebagai bagian dari template source, bukan sekadar data yang diberikan kepada template. Dampaknya tergantung engine dan exposed objects; pada kondisi tertentu bisa menjadi server compromise.

Hands-on 6.1 - Arithmetic proof only

Minimal proof

```
GET /ssti?name={{7*7}} HTTP/1.1
Host: 127.0.0.1:8120
```

Jika response berisi Hello 49, ekspresi template dievaluasi server-side. Stop pada arithmetic proof. Module ini tidak membutuhkan payload RCE.

Remediation

Jangan membangun template source melalui string concatenation dari user input. Gunakan fixed template dan pass user values sebagai data; harden/sandbox template engine bila user-authored template memang requirement.

7. Insecure Deserialization

Deserialization menjadi berbahaya ketika untrusted serialized data diubah kembali menjadi object yang memiliki behavior, type, magic method, atau gadget chain yang tidak diantisipasi. Dampak bisa terjadi selama proses deserialization, sebelum application logic utama berjalan.

Kenapa tidak ada custom gadget lab lokal?

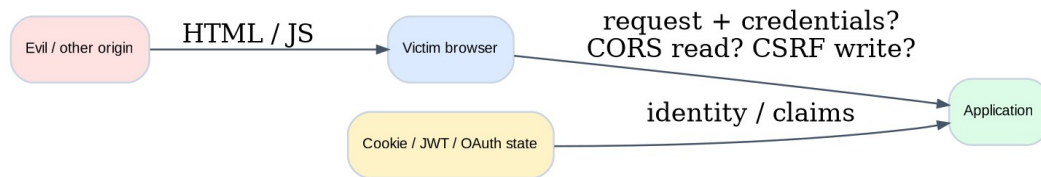
Gadget chain nyata sangat runtime-specific dan mudah berubah menjadi reusable arbitrary-code technique. Untuk hands-on, gunakan deliberately vulnerable PortSwigger labs sehingga environment dan objective terkontrol.

Hands-on 7.1 - PortSwigger guided lab

1. Buka topic Insecure Deserialization di Web Security Academy.
2. Mulai dari lab Apprentice yang menggunakan serialized session object.
3. Intercept session dengan Burp dan identifikasi format/fields sebelum mengubah apapun.
4. Ikuti objective lab untuk membuktikan trust terhadap serialized client data.
5. Tulis finding: attacker control -> deserializer -> unexpected object/state -> impact.

8. CORS vs CSRF - Jangan Disamakan

CORS mengontrol apakah JavaScript dari origin lain boleh membaca response. CSRF membahas apakah browser korban dapat dipaksa mengirim state-changing request dengan credentials. CORS bukan defense CSRF, dan CSRF masih mungkin terjadi walau attacker tidak dapat membaca response.



Browser menjadi mediator untuk CORS, CSRF, cookie, dan token behavior

Hands-on 8.1 - CORS origin reflection

1. Login ke local lab lewat /login supaya browser punya cookie session.
2. Dari directory labs, jalankan separate origin: `python -m http.server 8123 --bind 127.0.0.1`.
3. Buka `http://127.0.0.1:8123/csrf-attacker.html`.
4. Klik tombol Read local CORS endpoint.
5. Inspect request/response: Origin berbeda, server merefleksikan origin dan mengizinkan credentials, sehingga JavaScript dapat membaca response.

Separate local origin

```
cd modules/12-web-vulnerabilities-ii/labs
python -m http.server 8123 --bind 127.0.0.1
```

Hands-on 8.2 - CSRF state change

1. Masih dari attacker page pada port 8123, klik Submit local transfer.
2. Request POST menuju 127.0.0.1:8120 dan membawa session cookie karena browser mengelola credential.
3. Observe total_transferred bertambah walau aplikasi tidak memiliki CSRF token.

Subtle point

Origin memasukkan scheme + host + port. Site concept berbeda dan biasanya tidak memasukkan port. Lab ini membantu melihat bahwa cross-origin dan cross-site bukan sinonim.

Remediation

CORS: allow-list exact trusted origins, jangan reflection sembarangan, evaluasi credential requirement. CSRF: token yang terikat session, SameSite sesuai use case, Origin/Referer checks sebagai defense tambahan, dan hindari state change via GET.

9. JWT - Claims Tidak Sama dengan Trust

JWT payload mudah dibaca dan dimodifikasi oleh client. Security datang dari signature verification dan validation terhadap algorithm, issuer, audience, expiry, key selection, serta application authorization. Decode bukan verify.

Hands-on 9.1 - Deliberately broken alg=none verifier

1. GET /jwt-login untuk melihat token student.
2. Decode header/payload secara manual atau dengan Burp JWT tooling.
3. Buat token local training dengan role=admin menggunakan helper.
4. Kirim sebagai Bearer token ke /jwt-admin.
5. Dokumentasikan bahwa server menerima unsigned admin claims karena verifier sengaja menerima alg=none.

Local JWT proof

```
python labs/forge-jwt.py --user student --role admin

GET /jwt-admin HTTP/1.1
Host: 127.0.0.1:8120
Authorization: Bearer <TOKEN_DARI_HELPER>
```

Remediation

Gunakan maintained library, enforce algorithm allow-list dari server configuration, verifikasi signature sebelum mempercayai claims, validate iss/aud/exp/nbf, dan jangan membiarkan header JWT memilih trust root secara bebas.

10. OAuth / OpenID Connect

OAuth adalah delegated authorization framework; ketika dipakai untuk login biasanya ada OpenID Connect layer atau application-specific identity binding. Vulnerability sering muncul pada redirect_uri validation, state/nonce binding, scope, code/token leakage, dan perbedaan trust antara client dan provider.

Hands-on 10.1 - PortSwigger OAuth flow mapping

1. Buka OAuth authentication topic di Web Security Academy.
2. Pilih introductory Apprentice lab.
3. Di Burp HTTP History, gambar sequence: browser -> client app -> authorization server -> redirect/callback -> client session.
4. Identifikasi client_id, redirect_uri, state, scope, code/token, dan callback endpoint.
5. Kerjakan objective lab lalu tulis assumption yang gagal, bukan hanya final URL/payload.

Security principle

Exact redirect URI matching, strong state/nonce validation, PKCE pada flow yang relevan, minimal scope, dan issuer/audience validation mengurangi banyak implementation mistakes.

11. GraphQL Security

GraphQL memberi schema-driven query interface. Attack surface utamanya bukan "GraphQL itu insecure", tetapi exposure schema/introspection, missing object/field authorization, batching/alias abuse, excessive query cost, CSRF/content-type behavior, dan resolver-specific injection.

Hands-on 11.1 - Academy schema mapping

1. Buka PortSwigger GraphQL topic dan pilih beginner lab.
2. Temukan endpoint GraphQL dari browser/Burp.
3. Pelajari request shape: query, variables, operationName.
4. Gunakan introspection hanya jika lab memang mengizinkannya.
5. Map schema/object lalu uji authorization sesuai objective lab.
6. Catat perbedaan schema visibility vs actual access control.

12. NoSQL Injection

NoSQL injection dapat berupa syntax injection maupun operator injection. Karena NoSQL tidak punya satu query language universal, tester perlu memahami data type dan operator yang diproses backend.

Hands-on 12.1 - Local operator injection

Local operator-like proof

```
POST /nosql-login HTTP/1.1
Host: 127.0.0.1:8120
Content-Type: application/json

{"username":{"$ne":null},"password":{"$ne":null}}
```

Bandingkan dengan baseline student/student. Vulnerability lab sengaja memperlakukan object berisi \$ne sebagai kondisi truthy. Root cause-nya adalah server menerima query/operator structure dari untrusted JSON, bukan sekadar string value.

Remediation

Enforce schema/type validation, construct database query dari server-controlled operators, gunakan safe query builders/ODM correctly, dan jangan meneruskan arbitrary client objects sebagai query expression.

13. HTTP Host Header Attacks

Host adalah user-controlled request metadata. Application sering salah menganggapnya trusted ketika membuat absolute URL, password reset link, virtual-host routing, atau cache behavior.

Hands-on 13.1 - Password reset link poisoning concept

Change Host only

```
GET /reset-link HTTP/1.1
Host: attacker.invalid
```

Response local lab akan membangun reset_link menggunakan nilai Host. Tidak ada email atau korban nyata; proof cukup menunjukkan trusted absolute URL dibentuk dari user-controlled header.

Remediation

Gunakan configured canonical base URL, strict allowed-host validation, dan pahami reverse-proxy header behavior. Jangan membangun security-sensitive links dari raw Host/X-Forwarded-* tanpa trusted proxy model.

14. HTTP Request Smuggling / Desync

Request smuggling muncul ketika front-end dan back-end tidak sepakat tentang message boundaries atau request normalization. Ini adalah parser-differential class: satu byte stream menghasilkan dua interpretasi berbeda. Karena teknik ini dapat mengganggu shared connections dan user lain, hands-on wajib dilakukan di Web Security Academy.

No local raw desync lab

Module ini tidak memberikan reusable smuggling payload generator. Gunakan PortSwigger Academy karena infrastructure-nya memang sengaja dirancang untuk parser-disagreement testing tanpa menyentuh shared production connections.

Hands-on 14.1 - Academy methodology

1. Buka Request Smuggling learning material dan mulai dari introductory lab yang sesuai level.
2. Identifikasi front-end/back-end behavior dan protocol yang dipakai.
3. Gunakan Burp Repeater sesuai instruksi lab; jangan menjalankan high-volume probes.
4. Catat discrepancy: komponen A menganggap request selesai di mana, komponen B menganggap selesai di mana.
5. Tulis impact berdasarkan lab proof saja.

15. Web Cache Poisoning

Cache menyamakan request berdasarkan cache key. Poisoning terjadi ketika response dipengaruhi input yang tidak masuk cache key sehingga response attacker dapat disimpan dan diberikan ke request lain yang dianggap ekuivalen.

Hands-on 15.1 - Local unkeyed header demo

1. Reset local cache.
2. Kirim request /cache-home dengan X-Forwarded-Host: training.invalid.
3. Observe X-Cache: miss dan response mengandung training.invalid.
4. Kirim request normal tanpa header.
5. Observe X-Cache: hit dan response masih membawa value attacker-controlled dari request sebelumnya.

Local cache-key proof

```
POST /cache-reset HTTP/1.1
Host: 127.0.0.1:8120

GET /cache-home?cb=1 HTTP/1.1
Host: 127.0.0.1:8120
X-Forwarded-Host: training.invalid

GET /cache-home HTTP/1.1
Host: 127.0.0.1:8120
```

Remediation

Jangan biarkan unkeyed input mempengaruhi cacheable response. Include relevant variance dalam cache key, normalize trusted forwarding headers, atau disable caching untuk response yang bergantung pada user-specific/untrusted metadata.

16. Race Conditions

Race condition terjadi ketika rule dibuat sebagai check-then-act tanpa atomicity/locking yang tepat. Dua request paralel dapat sama-sama melewati check sebelum state diperbarui.

Hands-on 16.1 - Coupon redemption race

Run parallel local requests

```
curl -s -X POST http://127.0.0.1:8120/race/reset
python labs/race-demo.py
```

Observe berapa response sukses yang muncul sebelum state coupon_used stabil. Jalankan beberapa kali. Tujuan bukan speed, tetapi membuktikan invariant "coupon hanya sekali" rusak saat request diproses concurrently.

Remediation

Gunakan atomic database operation, transaction/unique constraint, idempotency key, or lock sesuai resource. Jangan mengandalkan “check lalu update” terpisah tanpa concurrency control.

17. Prototype Pollution

Prototype pollution adalah JavaScript-specific trust issue ketika attacker dapat menambah property ke object prototype yang kemudian diwarisi object lain. Vulnerability menjadi impactful ketika polluted property mencapai exploitable gadget seperti DOM sink atau server-side dangerous behavior.

Hands-on 17.1 - PortSwigger guided lab

1. Buka Prototype Pollution topic di Web Security Academy.
2. Mulai dari lab beginner yang menunjukkan source dan gadget.
3. Gunakan browser DevTools/Burp sesuai lab untuk mengidentifikasi source, polluted property, dan sink.
4. Stop pada objective lab; dokumentasikan chain source -> prototype -> gadget -> impact.

18. Business Logic Flaws

Logic flaw terjadi ketika aplikasi menerima state atau urutan yang secara teknis valid tetapi melanggar rule domain. Scanner sering gagal menemukan bug jenis ini karena butuh pemahaman workflow dan invariant.

Hands-on 18.1 - Negative quantity

Baseline vs unconventional input

```
GET /logic/checkout?price=100&quantity=1 HTTP/1.1
Host: 127.0.0.1:8120

# kemudian ubah satu business parameter
GET /logic/checkout?price=100&quantity=-2 HTTP/1.1
Host: 127.0.0.1:8120
```

Jika server menerima total negatif, business rule “quantity harus positif” tidak diverifikasi. Finding bukan “negative numbers are evil”; root cause adalah server gagal enforce invariant domain.

Remediation

Definisikan invariant business secara eksplisit dan enforce server-side pada setiap transition, termasuk unusual values, replay, duplicate step, concurrency, dan state yang tidak mungkin dari normal UI.

19. Guided Practice Matrix

Topik	Local lab	Academy follow-up
SSRF	/ssrf loopback internal service	SSRF beginner -> filter bypass / blind SSRF
XXE	POST /xml read harmless container hostname	XXE basic labs
SSTI	/ssti arithmetic proof	SSTI identification labs
Deserialization	-	Insecure deserialization labs
CORS / CSRF	separate local origin 8123	CORS + CSRF learning paths
JWT	alg=none local broken verifier	JWT labs
OAuth/OIDC	-	OAuth authentication labs
GraphQL	-	GraphQL labs
NoSQL	local operator-like input	NoSQL injection labs
Host header	/reset-link	Host header labs
Request smuggling	-	Request smuggling labs only
Cache poisoning	local unkeyed header cache	Web cache poisoning labs
Race conditions	local parallel redeem	Race condition labs

Prototype pollution	-	Prototype pollution labs
Business logic	negative quantity local	Business logic labs

20. Practical Challenge - Advanced Trust Boundary Assessment

Jalankan local lab tanpa membaca source. Objective: temukan minimal 7 behavior yang melanggar security/business assumption dan tulis finding berdasarkan evidence.

1. Start workspace dan lab.
2. Gunakan Burp HTTP History untuk map endpoints dari root page.
3. Untuk setiap endpoint, tulis expected security assumption sebelum testing.
4. Gunakan Repeater dan ubah satu variabel.
5. Temukan minimal 7 dari SSRF, XXE, SSTI, CORS, CSRF, JWT, NoSQL, Host header, cache poisoning, race, atau logic flaw.
6. Gunakan finding-note.py untuk minimal 5 finding.
7. Cleanup lab dan separate http.server.

Finding note example

```
python labs/finding-note.py module12-web-vulns-ii/05-findings/ssrf.md --title "Server-side request reaches internal-only service" --category "SSRF" --endpoint "GET /ssrf?url=..." --evidence "Server returned content from internal 127.0.0.1:8121 service." --impact "User-controlled destination crosses server-side network trust boundary." --remediation "Strict destination policy plus network egress controls."
```

Cleanup wajib

```
./labs/advanced-web-lab.sh cleanup
```

21. Knowledge Check

1. Apa perbedaan SSRF dengan open redirect?
2. Kenapa XXE dapat berubah menjadi SSRF pada sebagian parser/configuration?
3. Kenapa arithmetic evaluation sudah cukup sebagai proof SSTI awal?
4. Mengapa deserialization risk dapat terjadi sebelum business logic memakai object?
5. Apa perbedaan CORS dan CSRF?
6. Apa perbedaan origin dan site?
7. Kenapa decode JWT tidak sama dengan verify JWT?
8. Apa peran redirect_uri, state, dan PKCE dalam OAuth flow?
9. Kenapa GraphQL introspection bukan authorization control?
10. Bagaimana operator injection berbeda dari SQL string injection?
11. Mengapa Host header tidak boleh dianggap trusted?
12. Apa inti request smuggling sebagai parser differential?
13. Apa arti unkeyed input dalam cache poisoning?
14. Mengapa check-then-act rawan race condition?
15. Apa tiga komponen umum prototype pollution chain?
16. Mengapa business logic flaw sulit dideteksi scanner otomatis?

22. Cheat Sheet

Area	Minimal authorized proof	Root-cause focus
SSRF	server reads internal local endpoint	destination trust / egress
XXE	entity expands harmless local file	parser DTD/entity config
SSTI	{{7*7}} -> 49	user input becomes template source

CORS	other origin reads credentialed response	origin allow policy
CSRF	other origin triggers state change	request authenticity / token
JWT	modified unsigned claims accepted	signature + claim validation
NoSQL	operator object changes query behavior	type/schema/operator control
Host	absolute security link changes with Host	trusted canonical host
Cache	poisoned response served on later request	cache key mismatch
Race	multiple successes violate one-time rule	atomicity / concurrency
Logic	unconventional value breaks invariant	server-side business rule

Jangan jadikan cheat sheet sebagai payload catalog

Skill transferable adalah memahami trust boundary, parser behavior, state transition, dan proof minimal. Advanced web testing paling kuat ketika hypothesis datang sebelum payload.

23. Exit Criteria

- [] Local advanced lab dapat dibuild, dijalankan, dan di-cleanup tanpa exposure ke LAN.
- [] Saya dapat menjelaskan SSRF, XXE, SSTI, CORS, CSRF, JWT, NoSQL, Host header, cache poisoning, race, dan logic flaws dengan root cause.
- [] Saya memahami alasan deserialization, OAuth, GraphQL, request smuggling, dan prototype pollution dilatih di deliberately vulnerable Academy environment.
- [] Saya dapat membedakan browser security control dari server-side authorization.
- [] Saya dapat menjelaskan parser disagreement dan cache-key mismatch tanpa mengandalkan hafalan payload.
- [] Saya menyelesaikan minimal 5 Academy labs dari topik yang berbeda.
- [] Saya menghasilkan minimal 5 finding notes dari local challenge.
- [] Setiap finding memiliki baseline, changed input, observable effect, impact, dan remediation.
- [] Lab lokal sudah di-cleanup.

24. Referensi

PortSwigger - Server-side request forgery - SSRF theory and labs

<https://portswigger.net/web-security/ssrf>

PortSwigger - XML external entity injection - XXE theory and labs

<https://portswigger.net/web-security/xxe>

PortSwigger - Server-side template injection - SSTI theory and labs

<https://portswigger.net/web-security/server-side-template-injection>

PortSwigger - Insecure deserialization - Serialized object trust

<https://portswigger.net/web-security/deserialization>

PortSwigger - CORS - Cross-origin read policy

<https://portswigger.net/web-security/cors>

PortSwigger - CSRF - Cross-origin state-changing requests

<https://portswigger.net/web-security/csrf>

PortSwigger - JWT attacks - JWT validation flaws

<https://portswigger.net/web-security/jwt>

PortSwigger - OAuth authentication vulnerabilities - OAuth/OIDC implementation issues

<https://portswigger.net/web-security/oauth>

PortSwigger - GraphQL API vulnerabilities - GraphQL attack surface

<https://portswigger.net/web-security/graphql>

PortSwigger - NoSQL injection - Syntax/operator injection

<https://portswigger.net/web-security/nosql-injection>

PortSwigger - HTTP Host header attacks - Host header trust

<https://portswigger.net/web-security/host-header>

PortSwigger - HTTP request smuggling - Parser disagreement / desync

<https://portswigger.net/web-security/request-smuggling>

PortSwigger - Web cache poisoning - Cache key issues

<https://portswigger.net/web-security/web-cache-poisoning>

PortSwigger - Race conditions - Concurrency flaws

<https://portswigger.net/web-security/race-conditions>

PortSwigger - Prototype pollution - JavaScript prototype chain abuse

<https://portswigger.net/web-security/prototype-pollution>

PortSwigger - Business logic vulnerabilities - Application invariants and workflow flaws

<https://portswigger.net/web-security/logic-flaws>

OWASP - SSRF Prevention Cheat Sheet - Defensive SSRF design

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP - Cross-Site Request Forgery Prevention Cheat Sheet - CSRF defenses

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP - JSON Web Token Cheat Sheet for Java - JWT implementation considerations

https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

Next module

Module 13 fokus ke API Security: endpoint discovery, REST/JSON, BOLA/IDOR, authentication/authorization, mass assignment, rate limiting, JWT in API context, GraphQL API testing, business logic, dan API-specific evidence workflow.